DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Introducción al modelado VHDL de sistemas combinacionales y a la realización de Test-Benches**	**CÓDIGO:** **ATGNP2**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Ejercicios. Aprendizaje cooperativo.	TIPO:	No Presencial	DURACIÓN:	140 minutos

CALENDARIO:	Previo a Sesión Presencial S3	REQUISITOS:	Conocimientos adquiridos en Electrónica 2. Tutorial VHDL y ModelSim. Testbenches VHDL.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

Esta actividad tiene cuatro partes. En la primera descargará de Moodle los ficheros necesarios para llevarla a cabo y creará con ellos un proyecto ModelSim. En la segunda, tercera y cuarta trabajará con diferentes versiones de descodificadores para displays de 7 segmentos. Durante estas actividades se le pedirá que resuelva algunos ejercicios.

Aunque la actividad está clasificada en la modalidad de aprendizaje cooperativo, la dinámica de trabajo será la siguiente:

- Realice los ejercicios individualmente. Anote los resultados sobre el propio enunciado. Si tiene dudas puede consultar en el foro de dudas (preferentemente) o directamente a su profesor.
- Realice una puesta en común con su grupo de 4 estudiantes. Debe llevarse a cabo una discusión de los apartados en los que se encuentren discrepancias. Las dudas que subsistan deben ser anotadas y llevadas a la sesión presencial S3, en la cual serán resueltas por el profesor.

PARTE I. Preparación de la actividad

En esta actividad va a utilizar cuatro ficheros con código VHDL que debe descargar desde la plataforma Moodle; están asociados a la actividad ATGNP2 y se llaman:

- decodHex7seg.vhd
- decodBCD7seg_nBI.vhd
- decodBCDn7seg_nLT.vhd
- test_decodBCDn7seg_nLT.vhd

Realice las siguientes operaciones:

1. Descargue de Moodle el fichero ZIP que contiene todos los ficheros que va a utilizar en esta semana, y extráigalo en su cuenta de usuario[1].
2. Arranque el simulador ModelSim.

Procedimiento T.1: Creación de un proyecto con ficheros VHDL preexistentes

Para realizar esta actividad hay que crear un proyecto y, durante la creación, indicar que se desea añadir al mismo los cuatro ficheros que se han descargado desde Moodle. El nombre del proyecto debe ser *ATGNP2* y debe ubicarse (*Project Location*) en el directorio *ATGNP2/modelsim* –si no recuerda cómo se crea un proyecto, repase las instrucciones del procedimiento T.2 de la actividad AIP2). Al crear el proyecto aparece una ventana que ofrece la posibilidad de añadir ficheros VHDL ya existentes. Se utilizará para añadir los ficheros.

Realice las siguientes operaciones:

1. Cree el proyecto *ATGNP2* en el directorio *ATGNP2/modelsim*.

[1] En este texto se supone que está realizando la actividad en un PC del laboratorio. Si ha instalado el simulador ModelSim en un PC privado, sustituya, donde proceda, *"su cuenta de usuario"* por el directorio de trabajo que haya elegido para realizar la actividad.

2. Cuando aparezca la ventana de la figura 1, realice una doble pulsación sobre el icono *Add existing File*.

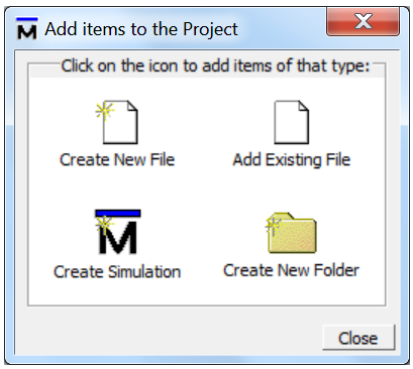


Figura 1.- *Add Existing File*

3. En la ventana que aparece (figura 2) pulse **Browse**.

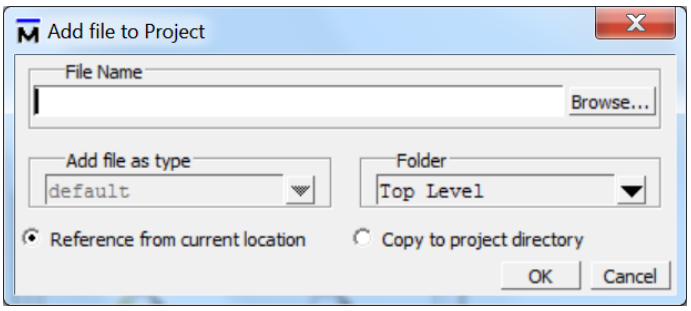


Figura 2.- Selección de ficheros (1)

4. En la nueva ventana (figura 3), seleccione con el ratón el fichero test_decodBCDn7seg_nLT.vhd y pulse **Abrir**.

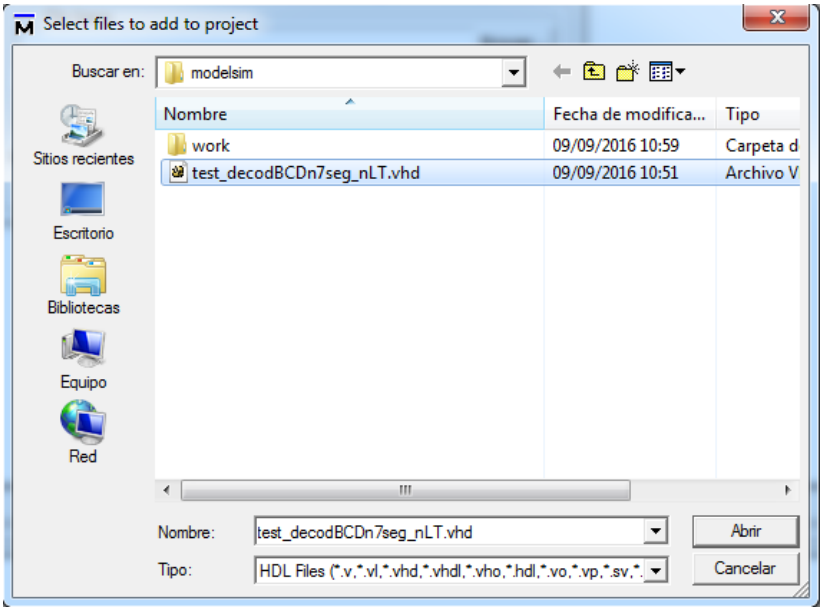


Figura 3.- Selección de ficheros (2)

5. Estará de vuelta en la ventana de la figura 2, pulse **OK** y observará que el fichero se añade a la pestaña **Project** de la ventana principal de ModelSim (figura 4).

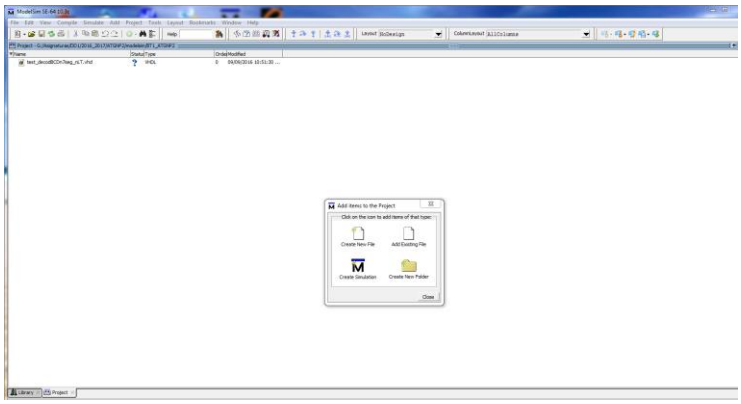


Figura 4.- Fichero añadido durante la creación del proyecto

6. Repita nuevamente el procedimiento para añadir los ficheros que se encuentran bajo el directorio ATGNP2/hdl. Para ello, cuando aparezca la ventana de la figura 3, debe moverse al directorio hdl pulsando en el icono 

7. Para terminar con el procedimiento, cierre la ventana de la figura 1 pulsando el botón **Close**.

PARTE II. Decodificador Hexadecimal a 7 segmentos

Realice las siguientes operaciones:

1. Abra el fichero **decodHex7seg.vhd**

Si analiza el código del modelo contenido en el fichero, observará que, a diferencia del modelo del decodificador BCD a 7 segmentos que se utilizó en el tutorial de ModelSim, en la sentencia CASE se especifica el valor de salida para las 16 combinaciones de entrada –en el modelo del decodificador BCD a 7 segmentos es indiferente (“XXXXXXX”) el valor de salida para las combinaciones de entrada mayores que 9.

2. Dibuje, sobre la representación de los *displays* de la figura 5, los segmentos que se iluminan para la combinación de entrada que se indica.

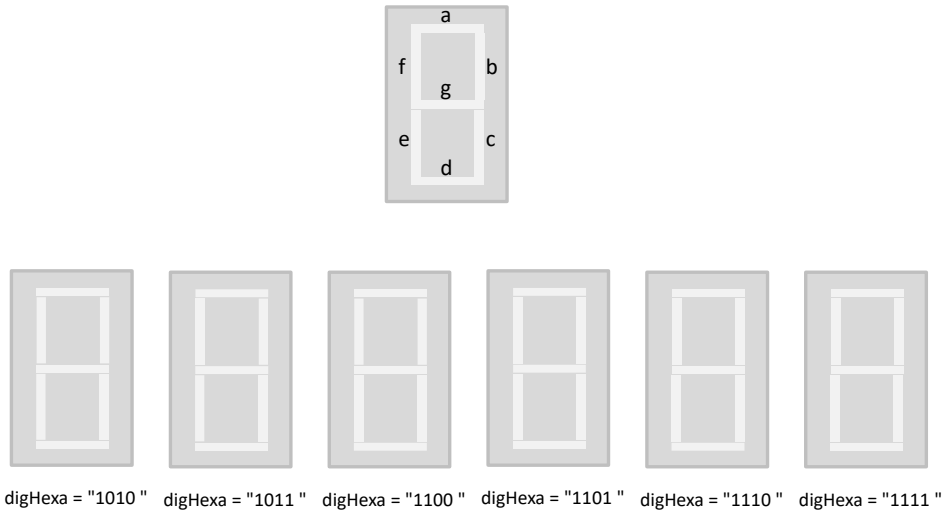


Figura 5

3. Compile el fichero **decodHex7seg.vhd** –si no recuerda cómo hacerlo, consulte el procedimiento T.9 de la actividad AIP2. El compilador detecta un error en el modelo.

   Si investiga el error, siguiendo el procedimiento de depuración de errores sintácticos (T.13 de AIP2), encontrará una ventana, como la de la figura 6, que le dará una información precisa sobre su causa: *"Case statement covers only 16 out of 6561 cases"*.

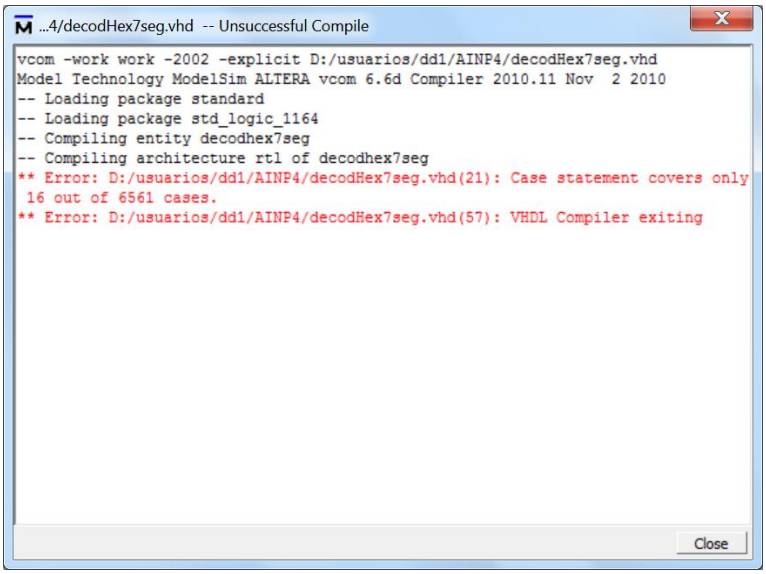


Figura 6

Efectivamente, las sentencias CASE VHDL tienen que considerar todos los posibles valores del *objeto* que evalúan; la entrada **digHexa** del modelo del decodificador tiene cuatro bits y es de tipo **std_logic_vector**, luego es un array de 4 elementos de tipo **std_logic** -tipo cuya declaración cuenta, como ya sabe, con 9 valores diferentes. El número de combinaciones diferentes que puede tener **digHexa** es, por tanto, $9^4 = 6561$.

En el código (erróneo) se evalúan las 16 combinaciones que pueden formarse con los valores que representan niveles lógicos ('0' y '1'), pero se han omitido 6545 combinaciones en las que aparecen valores *metalógicos* –combinaciones como, por ejemplo, "X-0L".

Para resolver este problema hay que añadir a la sentencia CASE el código que se muestra en la figura 7.

```
54          when others =>
55            segDisp <= "XXXXXXX";
```

Figura 7

Es muy importante que sea consciente de que esta fórmula, que también se empleaba en el modelo del decodificador BCD a 7 segmentos del tutorial de ModelSim, cumple aquí una función distinta:

- En el modelo del decodificador BCD a 7 segmentos indicaba que el valor de salida es indiferente cuando la combinación de entrada no es un dígito BCD y, simultáneamente, satisfacía el requisito de que la sentencia CASE considere todos los valores del objeto que evalúa.

- En el modelo del decodificador hexadecimal a 7 segmentos sólo se pone para cumplir la condición sintáctica de que la sentencia CASE evalúe exhaustivamente **digHexa**.

4. Añada el código de las líneas 54 y 55 de la figura 7 al modelo del decodificador hexadecimal a 7 segmentos para corregir el error. Salve el fichero y compílelo. Deberá obtener un mensaje, en la ventana **Transcript**, que le confirme el éxito de la compilación.

   Puede comprobar que la Declaración de Entidad y el Cuerpo de Arquitectura se han añadido a la librería Work.

PARTE III. Decodificador BCD a 7 segmentos con entrada de control de apagado

Un decodificador BCD a 7 segmentos puede contar con una entrada que apague todos los LED del *display*, independientemente del valor que haya en la entrada correspondiente al dígito BCD. Esta entrada suele denominarse BI (*Blanking Input*).

El fichero **decodBCD7seg_nBI.vhd** contiene el modelo VHDL de un decodificador con entrada de control de apagado (BI) activa a nivel bajo.

Realice las siguientes operaciones:

1. Abra el fichero **decodBCD7seg_nBI.vhd** con el editor de ModelSim.

   Hay algunos detalles del modelo que resulta necesario comentar. El primer detalle interesante es el nombre elegido para la entrada de control de borrado, **nBI**: es conveniente utilizar una *marca* en los nombres de las entradas y salidas activas a nivel bajo que ponga de manifiesto esta característica. Normalmente[2] se suele añadir una '**n**' delante o detrás del acrónimo o nombre que denota la función de la entrada o salida –siguiendo esta regla, nombres alternativos al elegido podrían ser **BIn** o **n_BI** o **BI_n**. Observe también que la entrada **nBI**, por ser de un bit, es de tipo **std_logic** (figura 8).

[2] Trabajando con ModelSim es conveniente que separe la 'n' del nombre con una barra ("n_" o "_n") para facilitar la legibilidad, porque ModelSim convierte todas las etiquetas del código a letras a minúsculas.

```
 9    entity decodBCD7seg_nBI is
10    port(
11         nBI:     in std_logic;
12         digBCD:  in std_logic_vector(3 downto 0);
13         segDisp: buffer std_logic_vector(6 downto 0)
14       );
15    end entity;
```

Figura 8

La sección de código más interesante de este modelo está en el proceso que modela el funcionamiento del circuito, porque muestra una estructura de uso muy frecuente en el modelado de sistemas combinacionales con entradas de control que habilitan (entradas de **enable**) o modifican (como la entrada **nBI** del decodificador) el funcionamiento *típico* del sistema. La estructura está formada por una combinación de sentencias IF y CASE; la expresión de selección de la sentencia IF condiciona, en el ejemplo, el funcionamiento normal o el apagado del *display*. En la figura 9 se representa, en *pseudocódigo*, esta estructura.

```
if {entrada de control activa} then
  funcionamiento  modificado

else
  funcionamiento típico
end if;
```

Figura 9

En el decodificador, el código del proceso que incluye esta fórmula es el que se muestra en la figura 10. La cláusula de selección **nBI = '0'** de la sentencia IF determina que para todas las combinaciones de entrada en las que **nBI** vale '0', todos los segmentos se apagan –observe el recurso sintáctico que se emplea para indicarlo, **(others => '0')**, en lugar de **"0000000"**; esta forma tiene la ventaja de que no hay que construir un *string* con tantos caracteres como bits tenga la salida. Para las combinaciones en las que **nBI** vale '1', una sentencia CASE define el valor de las combinaciones de salida del decodificador para construir los dígitos decimales –observe aquí que la fórmula **(others => 'X')** equivale al *string* **"XXXXXXX"**.

```
process(digBCD, nBI)
begin
  if nBI = '0' then
    segDisp <= (others => '0');

  else
    case digBCD is
     when "0000" =>
       segDisp <= "1111110";
     when "0001" =>
       segDisp <= "0110000";
     when "0010" =>
       segDisp <= "1101101";
     when "0011" =>
       segDisp <= "1111001";
     when "0100" =>
       segDisp <= "0110011";
```

```
      when "0101" =>
        segDisp <= "1011011";
      when "0110" =>
        segDisp <= "1011111";
      when "0111" =>
        segDisp <= "1110000";
      when "1000" =>
        segDisp <= "1111111";
      when "1001" =>
        segDisp <= "1110011";
      when others =>
        segDisp <= (others => 'X');
    end case;
  end if;
end process;
```

Figura 10

Realice las siguientes operaciones:

1. Compile el fichero **decodBCD7seg_nBI.vhd** y compruebe que el modelo queda almacenado en la librería **Work**.

Como ya sabe, las sentencias que componen el código de un proceso se ejecutan (desde la primera hasta la última) cuando las señales a las que es sensible el proceso cambian de valor –en jerga HDL: cuando sufren *eventos*- en el transcurso de una simulación.

Entender cómo se va a simular el código de un proceso es fundamental para progresar en el aprendizaje de las técnicas de modelado HDL; en el siguiente ejercicio se van a repasar los fundamentos del mecanismo de ejecución de los procesos en las simulaciones, utilizando como ejemplo el modelo del decodificador con entrada de control de apagado de los *displays*.

Realice las siguientes operaciones:

1. El cronograma de la figura 11 representa los estímulos de una simulación del decodificador con entrada **nBI**. Indique, en la tabla adjunta, los instantes del tiempo de simulación en que se ejecuta el proceso que modela el funcionamiento del decodificador, el número de línea donde se encuentra la sentencia de asignación de valor a la salida que se ejecuta en cada caso y, por último, el valor que se asigna a la salida.

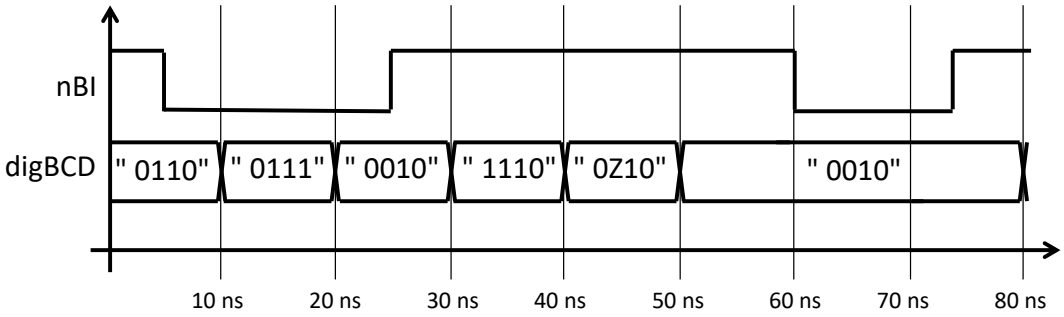
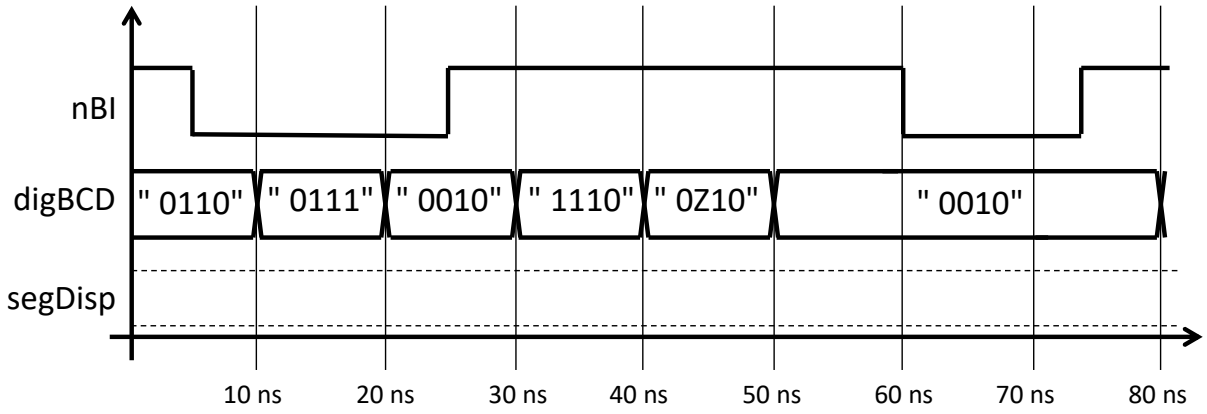


Figura 11

A modo de ejemplo: en T = 20 ns el proceso se ejecuta porque hay un evento en **digBCD**; como **nBI** vale ‘0’, se verifica la condición de la sentencia IF y se ejecuta la sentencia de asignación de la línea 22 de la figura 10, asignándose a todos los segmentos de salida un ‘0’.

Tiempo de Simulación	Número de Sentencia	Valor de Salida
T = 0 ns	**39**	**"1011111"**
T = 20 ns	**22**	**(others => '0')**
T = 75 ns		

2. Utilizando los datos de la tabla, complete el cronograma adjunto, en el que se repite la secuencia de estímulos del cronograma de la figura 11.



Test-Benches VHDL

Los simuladores VHDL ejecutan *test-benches*. Un *test-bench* VHDL esta formado por:

- ✓ Una Declaración de Entidad sin puertos que sirve para *identificar* el *test-bench*
- ✓ Un Cuerpo de Arquitectura en el que se declaran señales, que hacen las veces de estímulos para la simulación, se emplaza el modelo VHDL que se desea simular, conectándolo a las señales anteriormente mencionadas y se incluyen procesos que dirigen la simulación dando valores a los estímulos

Buena parte del código de un *test-bench* (todo el código salvo el correspondiente a los procesos que manejan los estímulos) puede generarse de manera rutinaria. Para ello basta con seguir las siguientes normas:

- ✓ Para la Declaración de Entidad simplemente hay que elegir un nombre; el nombre se construye habitualmente combinando el nombre del modelo que se simula con algún término o acrónimo que denote que se trata de la Declaración de Entidad de un *test-bench* (*test*, *tb*).

- ✓ El nombre del Cuerpo de Arquitectura debe denotar también que contiene la descripción de una prueba de simulación (*test*, por ejemplo)

- ✓ En el Cuerpo de Arquitectura, debajo de la cabecera, deben declararse tantas señales como puertos tenga el modelo que se desea simular, con el mismo nombre y tipo de datos que los puertos del modelo

- ✓ En el Cuerpo de Arquitectura debe incluirse una sentencia de emplazamiento que haga referencia al modelo que se desea simular; en dicha sentencia debe conectarse cada puerto con la señal homónima declarada en el *test-bench*

El código que se lista a continuación muestra una aplicación de las reglas anteriores para la realización de un *test-bench* para el modelo del decodificador BCD a 7 segmentos **decodBCD7seg_nBI**:

```
library ieee;
use ieee.std_logic_1164.all;

entity test_decodBCD7seg_nBI is
end entity;

architecture test of test_decodBCD7seg_nBI is
  signal nBI:     std_logic;
  signal digBCD:  std_logic_vector(3 downto 0);
  signal segDisp: std_logic_vector(6 downto 0);

begin
  dut: entity Work.decodBCD7seg_nBI(rtl)
       port map(nBI     => nBI,
                digBCD  => digBCD,
                segDisp => segDisp);
```

Para completar el *test-bench* basta con añadir a este código el del proceso que maneja los estímulos de la simulación. Estos procesos se caracterizan porque contienen sentencias WAIT y también porque, a diferencia de los utilizados para modelar hardware, carecen de lista de sensibilidad. En estos procesos la ejecución de sentencias de asignación de valor a los estímulos se intercala entre sentencias WAIT que determinan el tiempo que los estímulos mantienen los valores asignados. Por ejemplo, el siguiente proceso generaría, para las señales **nBI** y **digBCD**, la secuencia de valores que se observa en el cronograma de la figura 11:

```
process
  begin
    nBI    <= '1';   --Valores iniciales en T = 0 ns
    digBCD <= "0110";
    wait for 5 ns;   -- En T=5 ns se activa nBI
    nBI    <= '0';
    wait for 5 ns;   -- En T=10 ns digBCD = 0111
    digBCD <= "0111";
    wait for 10 ns;  -- En T=20 ns digBCD = 0010
    digBCD <= "0010";
    wait for 5 ns;   -- En T=25 ns se desactiva nBI
    nBI    <= '1';
```

```
    wait for 5 ns;   -- En T=30 ns digBCD = 1110
    digBCD <= "1110";
    wait for 10 ns;  -- En T=40 ns digBCD = 0Z10
    digBCD <= "0Z10";
    wait for 10 ns;  -- En T=50 ns digBCD = 0010
    digBCD <= "0010";
    wait for 10 ns;  -- En T=60 ns se activa nBI
    nBI    <= '0';
    wait for 15 ns;  -- En T=75 ns se desactiva nBI
    nBI    <= '1';
    wait;            -- Fin de simulación
  end process;
```

En el arranque de la simulación (en T = 0), el proceso anterior se ejecuta por primera vez, asignando los valores '1' y "0110" a las señales **nBI** y **digBCD**; la ejecución se suspende cuando se alcanza la primera sentencia WAIT FOR del proceso. Como consecuencia de la ejecución de la sentencia WAIT, el simulador apunta que el proceso deberá reanudar la ejecución de la siguiente sentencia (nBI <= '0'), cuando se alcance el tiempo de simulación T = 5 ns: el tiempo resultante de sumar al tiempo actual de ejecución, T = 0 ns, el tiempo especificado en la sentencia WAIT FOR (5 ns). Cuando el proceso reanuda su ejecución en T = 5 ns se asigna valor a **nBI** y se vuelve a suspender hasta T = 10 ns; en T = 10 ns se asigna el valor 0111 a **digBCD** y se suspende el proceso hasta que T = 20 ns: esta rutina de ejecución de sentencias de asignación y suspensión temporal del proceso se repite hasta que se alcanza la última sentencia (WAIT;) que suspende la ejecución del proceso definitivamente.

El código completo de un *test-bench* para el decodificador empleando el proceso de manejo de estímulos anterior quedaría:

```
library ieee;
use ieee.std_logic_1164.all;

entity test_decodBCD7seg_nBI is
end entity;

architecture test of test_decodBCD7seg_nBI is
  signal nBI:     std_logic;
  signal digBCD:  std_logic_vector(3 downto 0);
  signal segDisp: std_logic_vector(6 downto 0);

begin
  dut: entity Work.decodBCD7seg_nBI(rtl)
       port map(nBI     => nBI,
                digBCD  => digBCD,
                segDisp => segDisp);


  process
  begin
    nBI    <= '1';   --Valores iniciales en T = 0 ns
    digBCD <= "0110";
    wait for 5 ns;   -- En T=5 ns se activa nBI
    nBI    <= '0';
    wait for 5 ns;   -- En T=10 ns digBCD = 0111
    digBCD <= "0111";
    wait for 10 ns;  -- En T=20 ns digBCD = 0010
    digBCD <= "0010";
    wait for 5 ns;   -- En T=25 ns se desactiva nBI
    nBI    <= '1';
```

```
    wait for 5 ns;    -- En T=30 ns digBCD = 1110
    digBCD <= "1110";
    wait for 10 ns;   -- En T=40 ns digBCD = 0Z10
    digBCD <= "0Z10";
    wait for 10 ns;   -- En T=50 ns digBCD = 0010
    digBCD <= "0010";
    wait for 10 ns;   -- En T=60 ns se activa nBI
    nBI    <= '0';
    wait for 15 ns;   -- En T=75 ns se desactiva nBI
    nBI    <= '1';
    wait;             -- Fin de simulación
  end process;

end test;
```

Realice las siguientes operaciones:

1. El código que se adjunta a continuación sirve para generar una secuencia de estímulos que permite simular el funcionamiento del modelo del decodificador BCD a 7 segmentos. Dibuje en el cronograma los valores que tomarían las señales **nBI** y **digBCD** como consecuencia de la ejecución de dicho código en una simulación.

```
process
begin
  nBI    <= '1';
  digBCD <= "0000";

  wait for 10 ns;
  digBCD <= "0011";

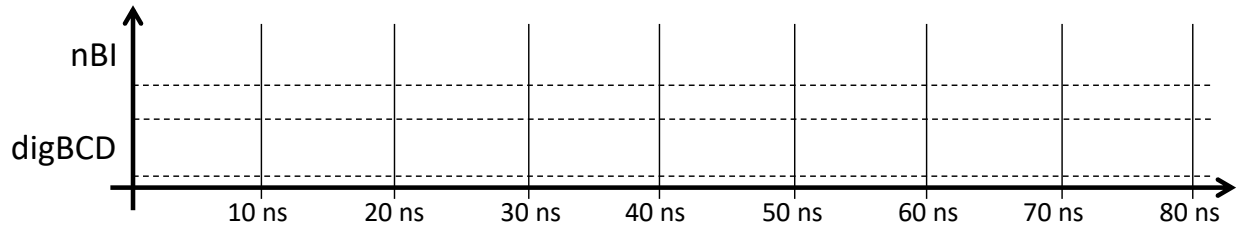
  wait for 10 ns;
  digBCD <= "1010";

  wait for 10 ns;
  nBI    <= '0';

  wait for 10 ns;
  digBCD <= "0110";

  wait for 10 ns;
  nBI    <= '1';

  wait for 10 ns;
  digBCD <= "0001";
  wait;
end process;
```



2. Realice el código de un proceso que genere una secuencia de estímulos para la simulación del decodificador BCD a siete segmentos como la que se muestra en el cronograma de la figura 12.

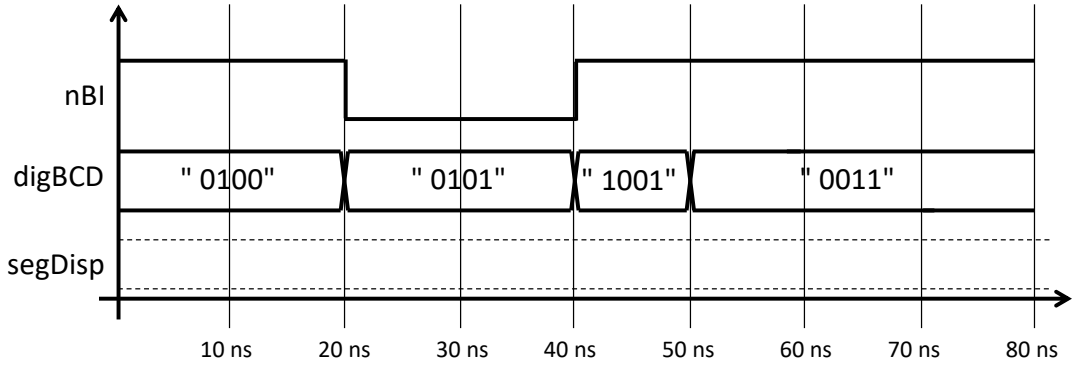


Figura 12

Escriba el código del proceso a continuación:

PARTE IV. Decodificador BCD a 7 segmentos con entrada de control de prueba de LEDs

El fichero decodBCDn7seg_nLT.vhd contiene el código incompleto del modelo de un decodificador BCD a 7 segmentos con las siguientes características:

- Salidas activas a nivel bajo
- Entrada de control, *Lamp Test*, que al activarse, a nivel bajo, enciende todos los LEDs del *display*;
- Cuando *Lamp Test* está inactiva, si el dato de entrada es un número entre 0 y 9, se activan las salidas necesarias para dibujar los dígitos decimales y, si tiene un valor entre 10 y 15, se apagan todos los segmentos.

Realice la siguiente operación:

1. Complete el código del decodificador, compile el fichero y compruebe que el modelo se añade a la librería de trabajo. Observe que, como ayuda, se han dispuesto comentarios VHDL en las líneas en las que tiene que editar código.

El fichero test_decodBCDn7seg_nLT.vhd contiene el código incompleto de un *test-bench* para el modelo del decodificador con entrada de *Lamp Test*

Realice la siguiente operación:

1. Complete el código del *test-bench* de modo que la secuencia de estímulos de simulación se corresponda con la mostrada en el cronograma de la figura 13. Observe que, como ayuda, se han dispuesto comentarios VHDL en las líneas en las que tiene que editar código.

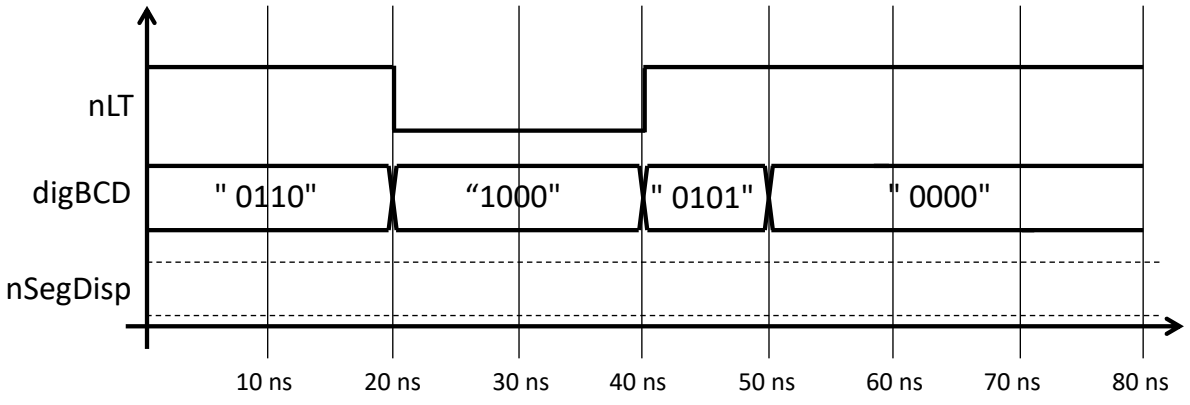


Figura 13